



Escuela de Ingeniería en Computadores

CE3201 - Taller de Diseño Digital

Laboratorio #2 - Documento de Diseño

Estudiantes:

Mario Gudiño Rovira
Michael Valverde Navarro

Profesor:

Luis Alonso Barboza Artavia

Semestre:

II Semestre 2025

Índice

1. Repositorio en Github	2
2. Propuestas de Diseño	2
2.1. Propuesta 1	2
2.2. Propuesta 2	4
2.3. Diseño elegido	5

1. Repositorio en Github

Enlace al repositorio de Github:

https://github.com/hazielgr/m_gudino_mvalverde_digital_design_lab_2025

2. Propuestas de Diseño

En este documento se presentan las propuestas de diseño para cada problema correspondiente del laboratorio 2.

Este problema corresponde a diseñar una ALU de manera estructural, la calculadora debe ser parametrizable, y debe realizar las siguientes operaciones de suma, resta, multiplicación, división, módulo, and, or, xor, shift left y shift right.

Para realizar esto se presentan las siguientes propuestas de diseño.

2.1. Propuesta 1

Para esta primera propuesta se considera realizar una ALU de forma que contenga un Adder de N-bits de entrada y N-bits de salida, y se le da una señal de varios bits que funciona como variable de control que permite especificar que función se va a realizar, esto se implementa utilizando un multiplexor, para las flags de NZCV estas se tienen pues dan información adicional sobre la salida de la ALU, se presenta el esquemático de como se ve la ALU implementada.

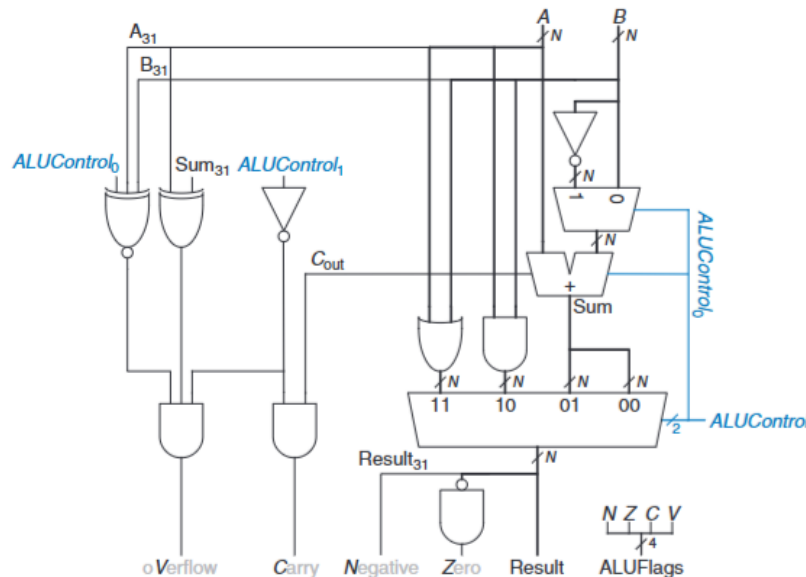


Figura 1: Esquemático de la ALU a implementar con la propuesta 1

2.1.1. Operaciones aritmeticas y logicas

El primer paso en el diseño de la ALU fue construir un sumador completo de 1 bit, utilizando únicamente operaciones lógicas básicas como AND, OR, XOR y NOT. Este bloque

recibe tres entradas: los operandos A y B de un solo bit, y un CarryIn (acarreo de entrada), generando como salidas la suma (S) y el CarryOut (acarreo de salida).

Para las operaciones logicas se investigan las tablas de verdad y se obtiene la tabla 1:

Cuadro 1: Tabla de verdad operaciones logicas

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Una vez se comprenden las operaciones basicas el siguiente paso era crear el sumador. Para el sumador se definió la tabla de verdad que se muestra en la figura 2

Cuadro 2: Tabla de verdad sumador completo 1 bit

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Con tabla de verdad se obtiene la expresion booleana a partir de ella y se representa de la siguiente manera.

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \wedge B) \vee (A \wedge C_{in}) \vee (B \wedge C_{in})$$

En cuanto al multiplicador y substractor se plantea reutilizar el adder de N bits, ya que ambas operaciones se basan en la suma, con la excepci3n de que un restador a uno de ambos n3meros debe cambiar de signo, y para el multiplicador se plantea hacer sumas consecutivas para dar el resultado esperado, se considera que esta opci3n es modular y proporciona un dise1o m1s limpio pues no se tienen que implementar modulos en System Verilog adicionales para cada una de las operaciones con las tablas de verdad b1sicas. En cuanto a operaciones como divisi3n, modulo, AND, OR, XOR, shift left y shift right se plantean utilizar los operandos ya implementados de System Verilog pues esto en la especificaci3n no esta restringido.

En cuanto a hardware a utilizar en la FPGA, se plantea utilizar switches (4 switches que permitan seleccionar el n3mero de entrada a la ALU en binario) y otros 4 switches que permita seleccionar las operaciones que se desean mostrar. Para dar paso a la operaci3n entre n3meros se utilizan dos push buttons, uno que permite la carga del primer bot3n, y otro bot3n distinto que permite la carga y resultado del siguiente n3mero.

Código (binario)	Operación
0000	ADD
0001	SUB
0010	MUL
0011	DIV
0100	MOD
0101	AND
0110	OR
0111	XOR
1000	SHL
1001	SHR

Cuadro 3: Propuesta de tabla de operaciones con sus códigos binarios dado por los switches de la FPGA

2.1.2. Resultado en FPGA

Se plantea el tener un botón o switch mapeado que sirva de Reset, con lo cual permitiría borrar los registros de la operación que se esta realizando en el momento y dar paso a una siguiente operación. La justificación para este reset es debido a que se considera que hay demasiadas operaciones y es fácil perderse al cambiar de una operación a otra.

Para dar resultados se utilizan los displays 7 segmentos, estos son exclusivos para resultados numéricos y se justifica el uso de LEDs para indicar las flags de negativo (N), cero (Z), carry (C) y overflow (V).

Para ver los resultados en la practica y el funcionamiento de la ALU se quiere utilizar la FPGA de la siguiente manera:

- **Switches:** se utilizarán para ingresar los operandos A y B de N bits, así como las señales de selección de operación (op). Esto permite elegir los valores de entrada y la operación que se desea ejecutar.
- **Botones:** se emplearán para cargar los operandos y activar la ejecución de la operación en la ALU.
- **LEDs:** se destinarán a la visualización de las banderas de estado (Carry, Zero, Overflow y Negativo), permitiendo verificar en tiempo real cómo cambia el estado interno de la ALU según la operación realizada.
- **Displays de 7 segmentos:** se utilizarán para mostrar el resultado de la operación ejecutada, permitiendo una lectura directa de la salida de la ALU en formato decimal.

2.2. Propuesta 2

Para la siguiente propuesta de diseño, se plantea una ALU que tenga un Adder de N bits, un Subtractor de N bits, Divisor de N bits, todos de forma separada para garantizar la completa implementación de estas operaciones, sin embargo se tiene en cuenta que este diseño no es modular y puede tomar más tiempo al desarrollarse, también se plantea que cada

operación utilice flags (N,Z,C,V) de manera local, es decir el sumador tiene sus variables de flags, el restador tiene sus variables de flags por aparte y así sucesivamente con el resto de operaciones, esto se justifica que es por simplicidad de los diseñadores para hacer pruebas y saber si los resultados son acertados o correctos pero se tiene en cuenta que esta implementación puede ser tediosa al escalar la ALU conforme pasa el tiempo. En cuanto a hardware se pensaba en utilizar los switches para ingresar los números a la ALU, utilizar 4 switches para ingresar un número, y otros 4 switches para ingresar el otro número, sin embargo la FPGA solo cuenta con 10 switches dejando únicamente 2 switches para seleccionar la operación a realizar y no son suficientes para esto.

2.3. Diseño elegido

En cuanto al diseño elegido se eligió la primera propuesta pues se considera que es una idea bien planteada, modular y mucho más madura, y más simple de realizar. En cuanto a la propuesta 2, se considera que falta más condiciones para considerarse un buen diseño y hay problemas que no se terminan de resolver y tampoco quedan claros como abordarse como es el caso del selector de operaciones.